Vigno Smart Class

Web Platform — Full Technical & Workflow Documentation

Stack: MERN (MongoDB, Express, React, Node.js) + AWS S3 / CloudFront + Razorpay

Theme: An education content platform where an admin builds Boards → Classes → Subjects → Modules → Chapters → Files (PDF, video, paid game files, etc.), with login, role control, payments, and Steam-style file protection adapted for the web.

Audience: Developers and project stakeholders. Written in simple language with tree charts, tables and step-by-step workflows.

Table of Contents

1. What We Are Building (Plain Language)

Vigno Smart Class is a website where school learning material is organised neatly, like folders inside folders. A logged-in admin uploads and manages all the content. Normal users log in to view free material, and pay to unlock premium material (such as special PDFs, video lessons, or downloadable game files).

Think of it like a digital school library combined with a small shop:

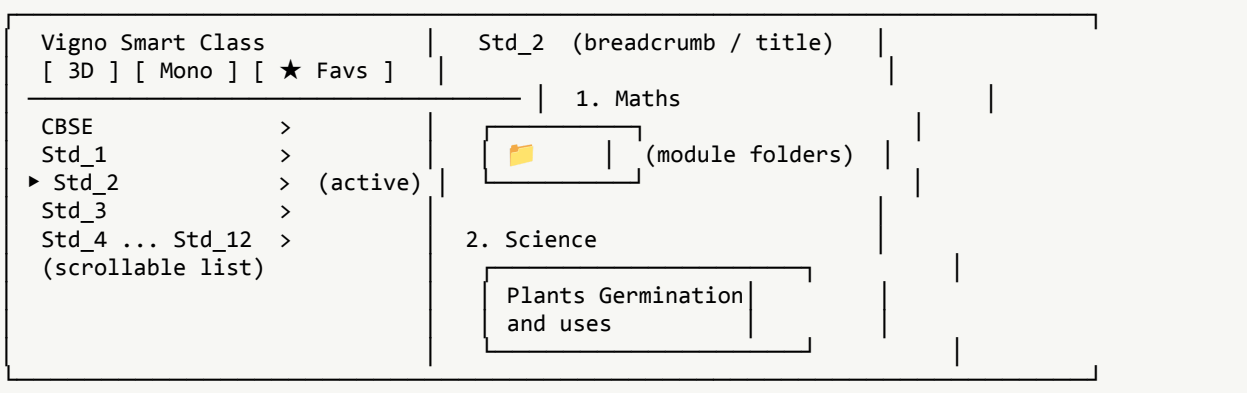
- The left side lets you pick a Board (e.g., CBSE) and a Class (Std_1 to Std_12).
- The main area shows Subjects (Maths, Science, ...). Inside each subject are folder cards called Modules.
- Open a module and you find Chapters / sub-folders, and inside those are the actual files.
- Some files are free; some are paid. Paid files stay locked until the user buys them.

The admin can create unlimited modules, chapters and files. Everything is controlled from the admin side — there is no separate admin login page; the system simply knows who is an admin.

2. The Screen Layout (from your screenshot)

The home screen has two main parts: a fixed left sidebar for navigation and a large right content area for the selected class. Below is the layout broken down.

Wireframe of the home screen:



2.1 Parts of the screen

UI Element	What it does	How it is built
App title + logo	Branding at top-left.	Static React component.
Tabs: 3D / Mono / Favs	Switch between 3D content, normal (Mono) content, and the user's favourites.	React tab state; 'Favs' reads the user's saved list from the API.
Board selector (CBSE)	Expandable item to choose an education board.	Data from the 'boards' collection; click loads its classes.
Class list (Std_1–Std_12)	Scrollable, selectable list. Active class is highlighted.	Mapped from the 'classes' collection; selected id stored in app state.

UI Element	What it does	How it is built
Breadcrumb / title (Std_2)	Shows where the user currently is.	Derived from the selected path (Board → Class).
Subject sections (Maths, Science)	Numbered groups inside the class.	From the 'subjects' collection, ordered by an 'order' field.
Folder cards (Modules)	Clickable cards that open a module's contents.	From the 'modules' collection; image/icon + title.
File view (inside a module)	Lists chapters and files; paid files show a lock + Buy button.	From 'chapters' and 'contents' collections; lock state from purchase check.

The dark red-to-orange gradient and rounded cards are styled with Tailwind CSS so the look matches your screenshot exactly and stays responsive on different screen sizes.

3. Technology Stack (MERN + supporting tools)

Your main stack is MERN. Around it we add a few well-known, free or low-cost tools for storage, payments, security and speed. Each choice is explained in simple terms.

3.1 Core stack

Layer	Technology	Why we use it (simple reason)
Frontend (what users see)	React + Vite	Fast, component-based UI; Vite gives quick builds and reloads.
UI styling	Tailwind CSS	Quickly build the dark gradient + card layout; responsive by default.
Client routing	React Router	Move between pages (login, class view, module view) without reloads.
Client data/cache	TanStack Query (React Query)	Fetches and caches API data so screens load fast and stay fresh.
Global state	Redux Toolkit (or Zustand)	Holds login status, selected class, theme, etc.
Backend (server)	Node.js + Express.js	Handles API requests, login, uploads, payments, security checks.
Database	MongoDB + Mongoose (Atlas)	Flexible 'documents' fit the folder-inside-folder content tree well.
File storage	AWS S3 (encrypted)	Stores PDFs, videos and game files safely and cheaply.
Content delivery (CDN)	AWS CloudFront	Serves files fast worldwide using short-lived signed URLs.
Payments	Razorpay	India-friendly gateway for selling paid content; supports cards, UPI, wallets.

Budget alternative: instead of AWS, you can use Cloudinary or Bunny.net for media + CDN and host the server on Render/Railway. The design stays the same; only the storage provider changes.

3.2 Helper libraries

Purpose	Library	Note
Password hashing	bcrypt	Never store plain passwords.
Login tokens	jsonwebtoken (JWT)	Proves who is logged in; kept in secure cookies.
File uploads	Multer	Receives uploaded files before sending to S3.
Input checking	Zod or Joi	Blocks bad/dangerous data.
Video streaming	HLS.js + Video.js/Plyr	Smooth, adaptive video playback.
Email/OTP	Nodemailer + (Twilio optional)	Verify accounts and send 2FA codes.

4. System Architecture (Big Picture)

Here is how the pieces talk to each other when a user uses the site.

Request flow:

```
       [ User's Browser – React app ]
                 │  (HTTPS only)
                 ▼
       [ Express API server (Node.js) ]
         │            │            │
         ▼            ▼            ▼
  [ MongoDB ]   [ AWS S3    ]  [ Razorpay ]
  accounts,     encrypted      payment
  content tree, files          checkout
  purchases          │
                     ▼
           [ CloudFront CDN ]
         serves files via SIGNED URLs
         (short-lived, per-user, expire fast)
```

Plain meaning: the browser only ever talks to our Express server over HTTPS. The server is the 'gatekeeper' — it checks the database for who you are and what you own, then either streams a file through a temporary signed CDN link, or refuses. Files are never exposed by a public, permanent link.

5. Content Hierarchy (the folder tree)

All content follows one clear tree. The admin can add unlimited items at the Module, Chapter and File levels.

Content tree:

```
Board (e.g., CBSE)
└── Class (Std_1 … Std_12)
    └── Subject (Maths, Science, …)   ← the numbered sections
        └── Module (folder card)      ← admin creates unlimited
            └── Chapter / Sub-folder   ← admin creates unlimited
                └── Content file       ← PDF / video / game file / etc.
                    • free   → open to any logged-in user
                    • paid   → locked until purchased
```

The first three folders you mentioned simply map to the first Subjects/Modules created at launch; nothing in the design limits the number — the admin grows the tree freely.

6. Database Design (MongoDB Collections)

Each level of the tree is a collection. A child stores the id of its parent, which keeps the tree fast to query.

Collection	Key fields	Purpose
users	name, email, passwordHash, role ('user'|'admin'), isVerified, twoFAEnabled, createdAt	Accounts and roles.
boards	name, order	Top level (CBSE, etc.).
classes	name (Std_2), boardId, order	Classes under a board.
subjects	name, classId, order	Maths, Science… sections.
modules	title, subjectId, icon/image, order	The folder cards.
chapters	title, moduleId, order	Sub-folders inside a module.
contents	title, chapterId, type (pdf/video/game/other), s3Key, isPaid, price, sizeBytes	The actual files.
purchases	userId, contentId, amount, razorpayPaymentId, status, purchasedAt	Who owns which paid file (the 'ownership' record).
favorites	userId, contentId	User's starred items (Favs tab).
auditlogs	actorId, action, targetId, ip, time	Security trail of admin actions.

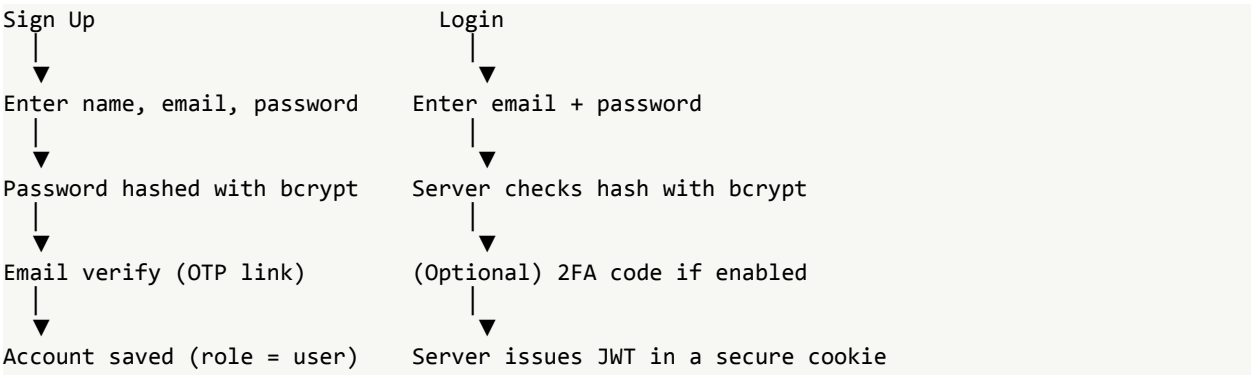
Optimization tip: we add database indexes on the parent-id fields (e.g., modules.subjectId) and on purchases (userId+contentId) so lookups are instant even with lots of data.

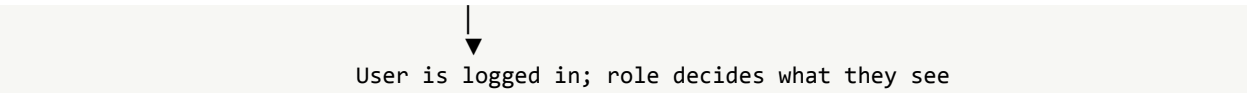
7. Login & Account System

There is one single login page for everybody — no separate admin button. The system decides your powers from your 'role' field after you log in.

7.1 Sign-up / login workflow

Account flow:





The login token (JWT) is stored in an httpOnly + Secure + SameSite cookie. This means JavaScript and attackers cannot read it, which blocks most token-theft attacks. A short-lived access token plus a longer refresh token keeps users logged in safely.

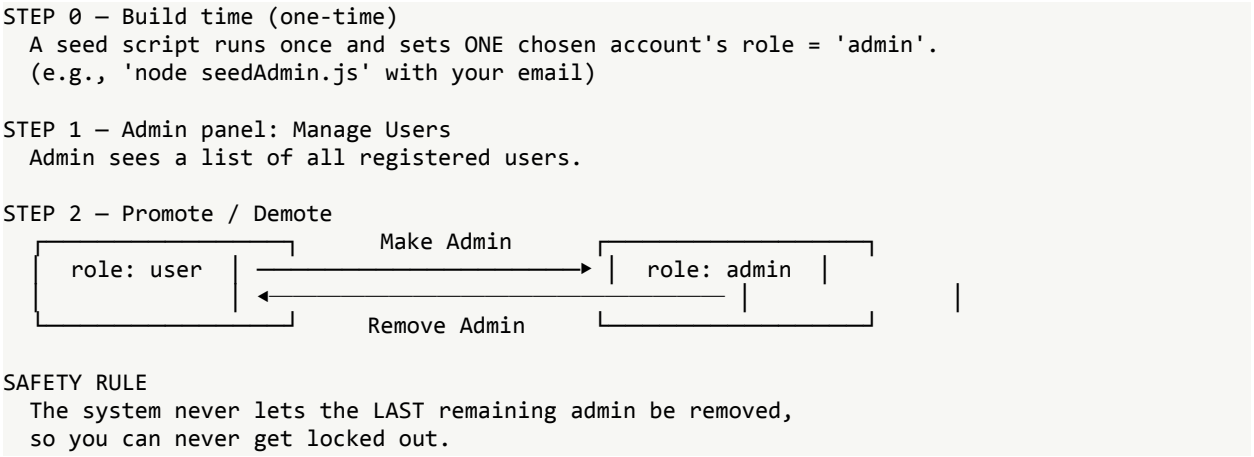
7.2 Roles & permissions

Role	Can do	Cannot do
User	Log in, browse, view free content, buy & view purchased content, save favourites.	Create/edit content, manage other users.
Admin	Everything a user can, PLUS create/edit/delete all content, upload files, set prices, view sales, and promote/demote other users.	—

8. How Admins Are Created & Managed

You asked for: (1) the very first admin assigned during building, and (2) existing admins able to make or remove other admins. Here is exactly how that works.

Admin management flow:



Every promote/demote action is written to the auditlogs collection (who did it, to whom, when, from which IP). This gives you a clear security trail.

9. File Security — Steam-Style Protection, Adapted for the Web

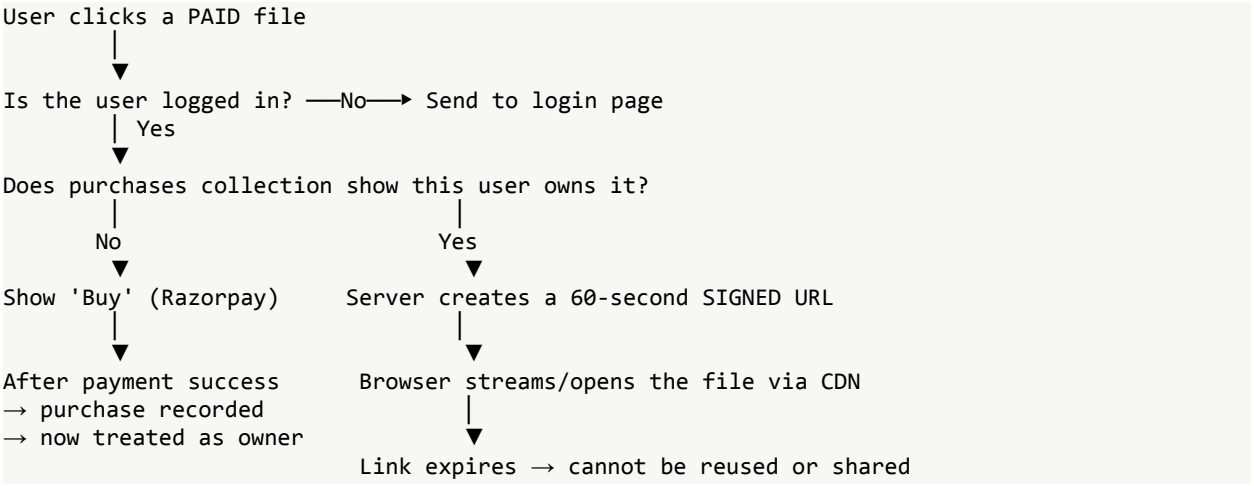
Steam protects native game files with encryption (CEG) plus an ownership check against its servers. A website cannot run Steam's executable trick, The goal is identical: a copied link or file is useless unless the server confirms you truly own it.

9.1 Steam idea → our web equivalent

Steam does this	We do this on the web	Tech used
Ships the game encrypted (CEG)	Store every file encrypted at rest	AWS S3 with SSE-KMS encryption
Checks account login first	Require JWT login before any file request	JWT + httpOnly cookies
Asks servers 'do you own this?'	Server checks the purchases collection before serving paid files	Express middleware + MongoDB
Hands a key only to the owner	Issue a short-lived signed URL only to the owner	CloudFront signed URLs / signed cookies
Decrypts in memory only	Stream content; no permanent public link; link expires in minutes	Time-limited URLs + HLS streaming
Trade/Market holds, 2FA	Optional 2FA, device/session limits, download caps	TOTP/OTP + rate limiting
Anti-tamper / watermark	Watermark PDFs with the buyer's email/id to trace leaks	PDF watermarking (server-side)
(Strongest) encrypted streaming	Optional Widevine DRM for premium videos	Widevine / EME (upgrade option)

9.2 Workflow: opening a paid file

Paid-file access flow:



Because the signed URL lasts only a minute and is tied to the request, a user cannot copy the link and share it. Even if they did, it would already be expired. The original file in S3 has no public address at all.

10. Payments (Razorpay) Workflow

Selling a paid file uses a safe two-step verify so nobody can fake a payment.

Payment flow:

```
1. User clicks Buy
2. Server creates a Razorpay 'order' (amount, currency)
```

```
3. Razorpay checkout opens in the browser; user pays
4. Razorpay sends back a payment id + signature
5. SERVER verifies the signature (secret key) ── invalid ─► reject
        │ valid
        ▼
6. Save a row in 'purchases' (user owns this content)
7. A Razorpay webhook double-confirms payment in the background
8. User can now open the file (see Section 9.2)
```

The signature check in step 5 is the key: the browser cannot forge it because only our server holds the secret key. The webhook in step 7 is a backup confirmation in case the browser closes early.

11. Security Technologies (full list)

Every tool below has one job: keep accounts, payments and files safe.

Area	Technology / Practice	What it protects against
Transport	HTTPS / TLS everywhere	Eavesdropping on data in transit.
Passwords	bcrypt hashing (salted)	Stolen-database password leaks.
Sessions	JWT in httpOnly+Secure+SameSite cookies	Token theft via JavaScript / XSS.
Access control	Role-based middleware (RBAC)	Users reaching admin actions.
Brute force	express-rate-limit	Password guessing, API abuse.
Input safety	Zod/Joi validation + express-mongo-sanitize	Injection and malformed data.
Headers	Helmet	Clickjacking, MIME sniffing, etc.
Cross-site	CORS rules + CSRF tokens	Requests from unknown sites.
Files at rest	S3 SSE-KMS encryption	Stolen raw files being readable.
File access	CloudFront signed URLs (short-lived)	Link sharing / hotlinking / piracy.
Ownership	purchases check before every paid access	Non-buyers opening paid content.
Accounts	Optional 2FA (TOTP/OTP)	Account takeover.
Tracing leaks	Per-user PDF watermarking	Identifying who leaked a file.
Premium video	Widevine DRM (optional)	Screen/stream ripping of videos.
Monitoring	Audit logs	Untracked admin/account changes.

12. Optimization Technologies (speed & scale)

These keep the site fast and cheap to run, even with many users and large files.

Area	Technology / Practice	Benefit
Global delivery	CloudFront CDN	Files load fast from a nearby server.

Area	Technology / Practice	Benefit
Client caching	TanStack Query	Avoids refetching the same data.
Smaller bundles	Code splitting + lazy loading (React)	Faster first page load.
Images	Lazy loading + compression/WebP	Less data, quicker pages.
Video	HLS adaptive streaming	Smooth playback on any connection.
Database	Indexes + pagination	Instant queries on big collections.
Server cache	Redis (optional)	Caches hot data and sessions.
Responses	gzip / brotli compression	Smaller, faster API responses.
Search	Debounced search + text index	Quick search without overloading the server.
Uptime	PM2 + load balancing	App stays up and handles traffic spikes.

13. Development Plan (Step by Step)

Build in phases so each part can be tested before the next. Recommended order:

1. Project setup: create React (Vite) frontend and Express backend; connect MongoDB Atlas; set environment variables.
2. Database models: build all Mongoose schemas (users, boards, classes, subjects, modules, chapters, contents, purchases, favorites, auditlogs).
3. Authentication: sign-up, email verification, login, JWT cookies, password hashing, optional 2FA.
4. Roles & admin seeding: RBAC middleware; one-time seed script to assign the first admin; Manage-Users screen with promote/demote and the last-admin safety rule.
5. Content management (admin CMS): create/edit/delete boards → classes → subjects → modules → chapters; reorder with the 'order' field.
6. Frontend layout: build the exact screen from the screenshot — sidebar, tabs (3D/Mono/Favs), class list, subject sections, folder cards — with Tailwind.
7. File uploads: Multer endpoint → push to encrypted S3; save metadata (type, size, isPaid, price) in 'contents'.
8. File security: signed-URL service; ownership/purchase middleware; PDF watermarking; (optional) Widevine for video.
9. Payments: Razorpay order + signature verify + webhook; record purchases; unlock content.
10. Optimization: add CDN, indexes, caching, lazy loading, compression.
11. Testing & deployment: unit/integration tests, security review, then deploy (frontend on a CDN host, backend with PM2, DB on Atlas).

14. Suggested Project Folder Structure

Folder layout:

```
vigno-smart-class/
├── client/                 (React + Vite frontend)
│   ├── src/
│   │   ├── components/      (Sidebar, Tabs, FolderCard, …)
│   │   ├── pages/           (Login, ClassView, ModuleView, Admin)
│   │   ├── store/           (Redux / Zustand)
```

```
│   │   └── api/              (axios + React Query hooks)
│   └── tailwind.config.js
└── server/                  (Node + Express backend)
    ├── models/              (Mongoose schemas)
    ├── routes/              (auth, content, files, payments, users)
    ├── middleware/          (auth, RBAC, rate-limit, validation)
    ├── services/            (S3, signedUrl, razorpay, watermark)
    ├── scripts/             (seedAdmin.js)
    └── server.js
```

15. Main API Endpoints (overview)

Method & Path	Who	Does
POST /auth/signup	Public	Create account (role=user).
POST /auth/login	Public	Log in, set JWT cookie.
POST /auth/2fa/verify	User	Verify 2FA code.
GET /classes/:id/tree	User	Get subjects→modules→chapters for a class.
POST /admin/modules	Admin	Create a module folder.
POST /admin/contents/upload	Admin	Upload a file to encrypted S3.
GET /contents/:id/access	User	Get a short-lived signed URL (after ownership check).
POST /payments/order	User	Create a Razorpay order for a paid file.
POST /payments/verify	User	Verify payment and record purchase.
GET /admin/users	Admin	List users.
PATCH /admin/users/:id/role	Admin	Promote/demote (with last-admin guard).

16. One-Paragraph Summary

Vigno Smart Class is a MERN web app where a single login serves both users and admins, and your role (set first by a seed script, then managed by admins) decides your powers. Admins build an unlimited folder tree of Boards → Classes → Subjects → Modules → Chapters → Files and upload PDFs, videos and paid game files. Files are stored encrypted in AWS S3 and only ever reach a user through a short-lived signed CloudFront link after the server confirms login and, for paid items, a recorded purchase via Razorpay — the same 'lock it, verify the owner, then unlock briefly' idea Steam uses, rebuilt with web tools. Helmet, bcrypt, JWT cookies, RBAC, rate limiting, validation and audit logs keep it secure, while a CDN, caching, indexes, lazy loading and compression keep it fast.